Git详解

版本控制

什么是版本控制

版本控制（Revision control）是一种在开发的过程中用于管理我们对文件、目录或工程等内容的修改历史，方便查看更改历史记录，备份以便恢复以前的版本的软件工程技术。

- 实现跨区域多人协同开发
- 追踪和记载一个或者多个文件的历史记录
- 组织和保护你的源代码和文档
- 统计工作量
- 并行开发、提高开发效率
- 跟踪记录整个软件的开发过程
- 减轻开发人员的负担，节省时间，同时降低人为错误

简单说就是用于管理多人协同开发项目的技术。

没有进行版本控制或者版本控制本身缺乏正确的流程管理，在软件开发过程中将会引入很多问题，如软件代码的一致性、软件内容的冗余、软件过程的事物性、软件开发过程中的并发性、软件源代码的安全性，以及软件的整合等问题。

> 多人开发就必须要使用版本控制，不然代价会很大

常见的版本控制工具

我们学习的东西，一定是当下最流行的！

主流的版本控制器有如下这些：

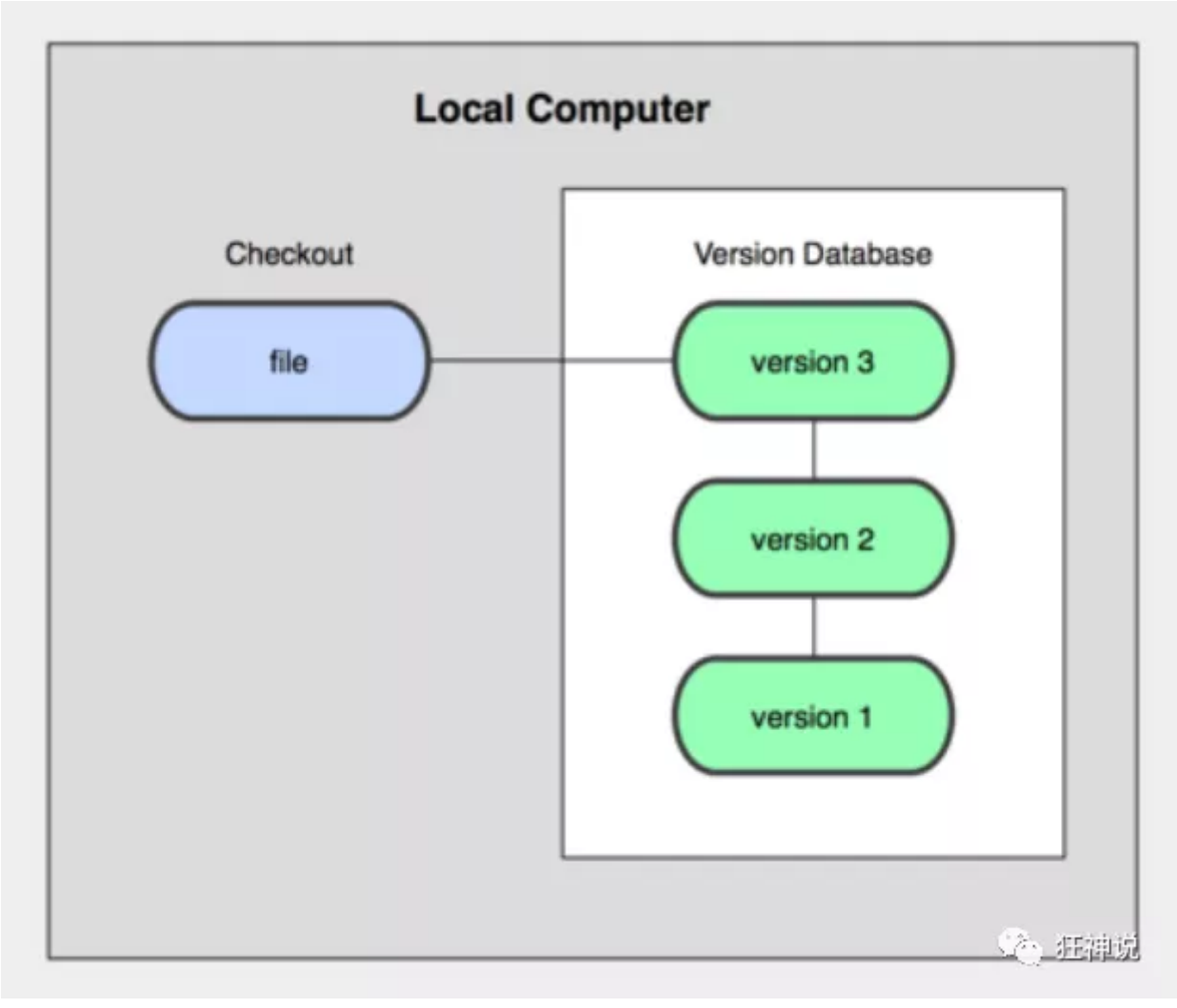
- **Git**
- **SVN**（Subversion）
- **CVS**（Concurrent Versions System）
- **VSS**（Micorosoft Visual SourceSafe）
- **TFS**（Team Foundation Server）
- Visual Studio Online

版本控制产品非常的多（Perforce、Rational ClearCase、RCS（GNU Revision Control System）、Serena Dimention、SVK、BitKeeper、Monotone、Bazaar、Mercurial、SourceGear Vault），现在影响力最大且使用最广泛的是Git与SVN。

版本控制分类

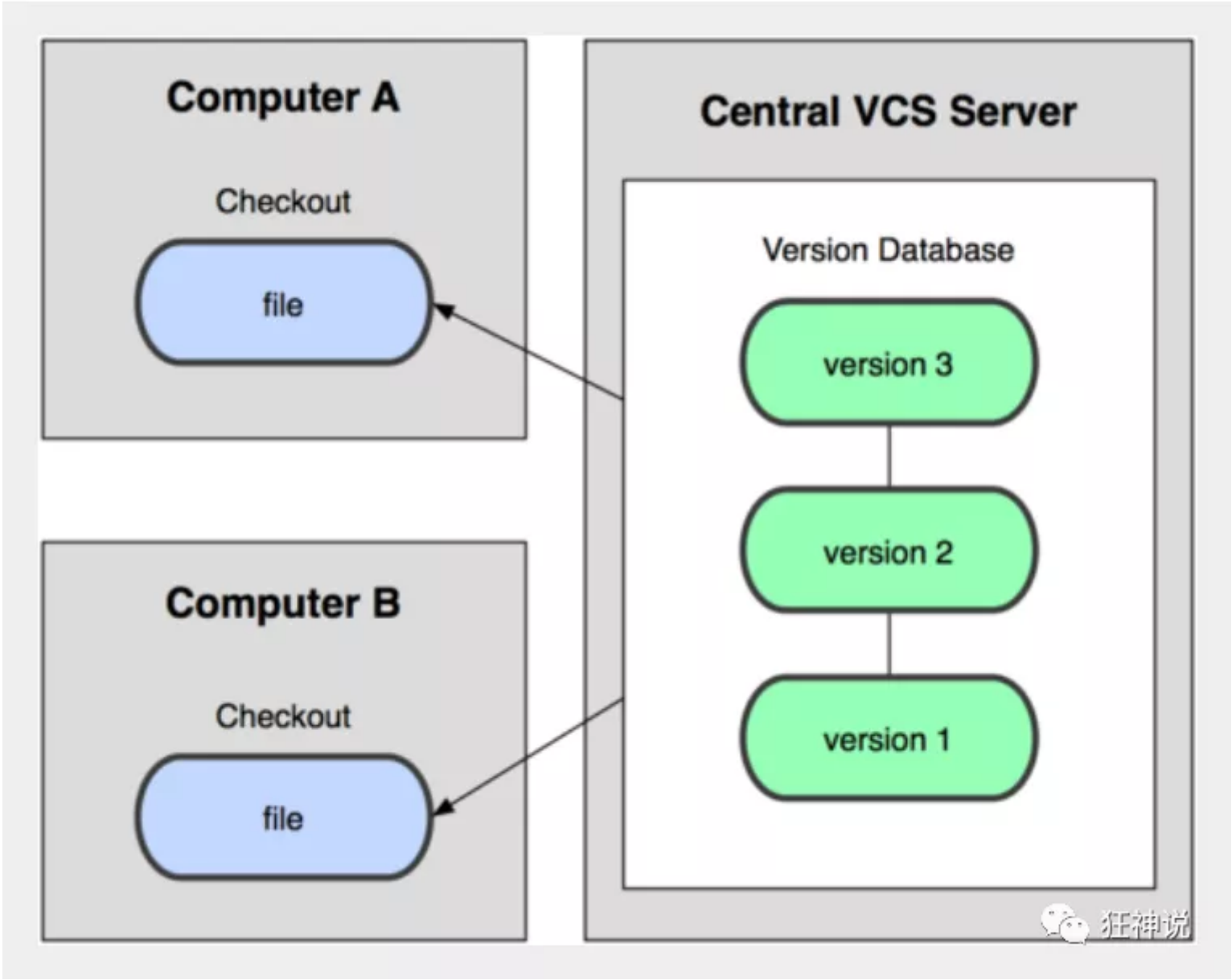
本地版本控制

记录文件每次的更新，可以对每个版本做一个快照，或是记录补丁文件，适合个人用，如RCS。



集中版本控制 SVN

所有的版本数据都保存在服务器上，协同开发者从服务器上同步更新或上传自己的修改



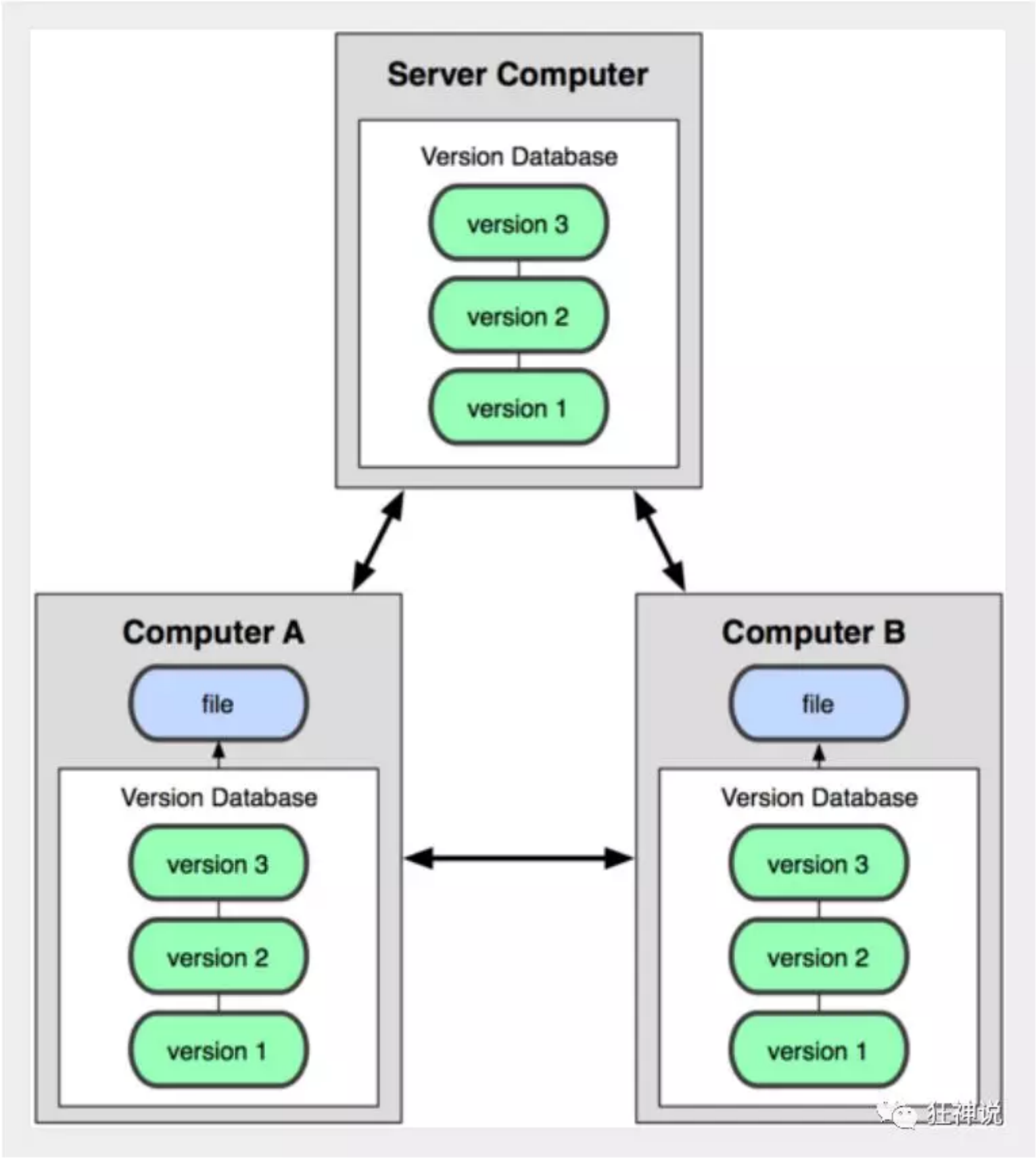
所有的版本数据都存在服务器上，用户的本地只有自己以前所同步的版本，如果不连网的话，用户就看不到历史版本，也无法切换版本验证问题，或在不同分支工作。而且，所有数据都保存在单一的服务器上，有很大的风险这个服务器会损坏，这样就会丢失所有的数据，当然可以定期备份。代表产品：SVN、CVS、VSS

分布式版本控制 Git

每个人都拥有全部的代码！安全隐患！

所有版本信息仓库全部同步到本地的每个用户，这样就可以在本地查看所有版本历史，可以离线在本地提交，只需在连网时push到相应的服务器或其他用户那里。由于每个用户那里保存的都是所有的版本数据，只要有一个用户的设备没有问题就可以恢复所有的数据，但这增加了本地存储空间的占用。

不会因为服务器损坏或者网络问题，造成不能工作的情况！



Git与SVN的主要区别

SVN是集中式版本控制系统，版本库是集中放在中央服务器的，而工作的时候，用的都是自己的电脑，所以首先要从中央服务器得到最新的版本，然后工作，完成工作后，需要把自己做完的活推送到中央服务器。集中式版本控制系统是必须联网才能工作，对网络带宽要求较高。

Git是分布式版本控制系统，没有中央服务器，每个人的电脑就是一个完整的版本库，工作的时候不需要联网了，因为版本都在自己电脑上。协同的方法是这样的：比如说自己在电脑上改了文件A，其他人也在电脑上改了文件A，这时，你们两之间只需把各自的修改推送给对方，就可以互相看到对方的修改了。Git可以直接看到更新了哪些代码和文件！

Git是目前世界上最先进的分布式版本控制系统。

Git历史

同生活中的许多伟大事物一样，Git 诞生于一个极富纷争大举创新的年代。

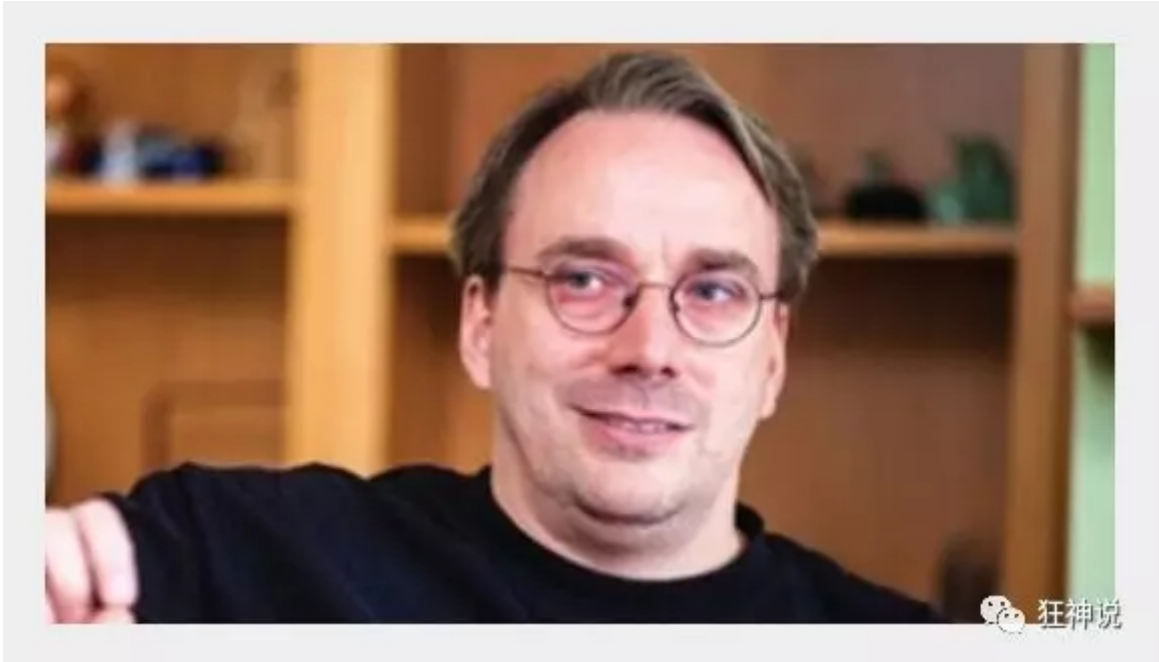
Linux 内核开源项目有着为数众广的参与者。绝大多数的 Linux 内核维护工作都花在了提交补丁和保存归档的繁琐事务上(1991 - 2002年间)。到 2002 年，整个项目组开始启用一个专有的分布式版本控制系统 BitKeeper 来管理和维护代码。

Linux社区中存在很多的大佬！破解研究 BitKeeper ！

到了 2005 年，开发 BitKeeper 的商业公司同 Linux 内核开源社区的合作关系结束，他们收回了 Linux 内核社区免费使用 BitKeeper 的权力。这就迫使 Linux 开源社区(特别是 Linux 的缔造者 Linus Torvalds)基于使用 BitKeeper 时的经验教训，开发出自己的版本系统。（2周左右！） 也就是后来的 Git！

Git是目前世界上最先进的分布式版本控制系统。

Git是免费、开源的，最初Git是为辅助 Linux 内核开发的，来替代 BitKeeper！



Linux和Git之父李纳斯·托沃兹（Linus Benedic Torvalds）1969、芬兰

Git环境配置

安装

- http://npm.taobao.org/mirrors/git-for-windows/
- https://git-scm.com/

常用的Linux命令

1）、cd : 改变目录。

2）、cd . . ：回退到上一个目录，直接cd进入默认目录

3）、pwd : 显示当前所在的目录路径。

4）、ls(ll): 都是列出当前目录中的所有文件，只不过ll(两个ll)列出的内容更为详细。

5）、touch : 新建一个文件 如 touch index.js 就会在当前目录下新建一个index.js文件。

6）、rm: 删除一个文件, rm index.js 就会把index.js文件删除。

7）、mkdir: 新建一个目录,就是新建一个文件夹。

8）、rm -r : 删除一个文件夹, rm -r src 删除src目录

```
rm -rf / 切勿在Linux中尝试！删除电脑中全部文件！
```

9）、mv 移动文件, mv index.html src index.html 是我们要移动的文件, src 是目标文件夹,当然, 这样写,必须保证文件和目标文件夹在同一目录下。

10）、reset 重新初始化终端/清屏。

11）、clear 清屏。

12）、history 查看命令历史。

13）、help 帮助。

14）、exit 退出。

15）、#表示注释

Git配置

所有的配置文件，其实都保存在本地！

查看配置 git config -l

```
MINGW64:/c/Users/Administrator
Administrator@Kuangshen MINGW64 ~
$ git config -l
core.symlinks=false
core.autocrlf=true
core.fscache=true
color.diff=auto
color.status=auto
color.branch=auto
color.interactive=true
help.format=html
rebase.autosquash=true
http.sslcainfo=D:/ProgramFiles/Git/mingw64/ssl/certs/ca-bundle.crt
http.sslbackend=openssl
diff.astextplain.textconv=astextplain
filter.lfs.clean=git-lfs clean -- %f
filter.lfs.smudge=git-lfs smudge -- %f
filter.lfs.process=git-lfs filter-process
filter.lfs.required=true
credential.helper=manager
core.editor="D:\\ProgramFiles\\Notepad++\\notepad++.exe" -multiInst -notabbar
-nosession -noPlugin
user.name=kuangshen
```

狂神说

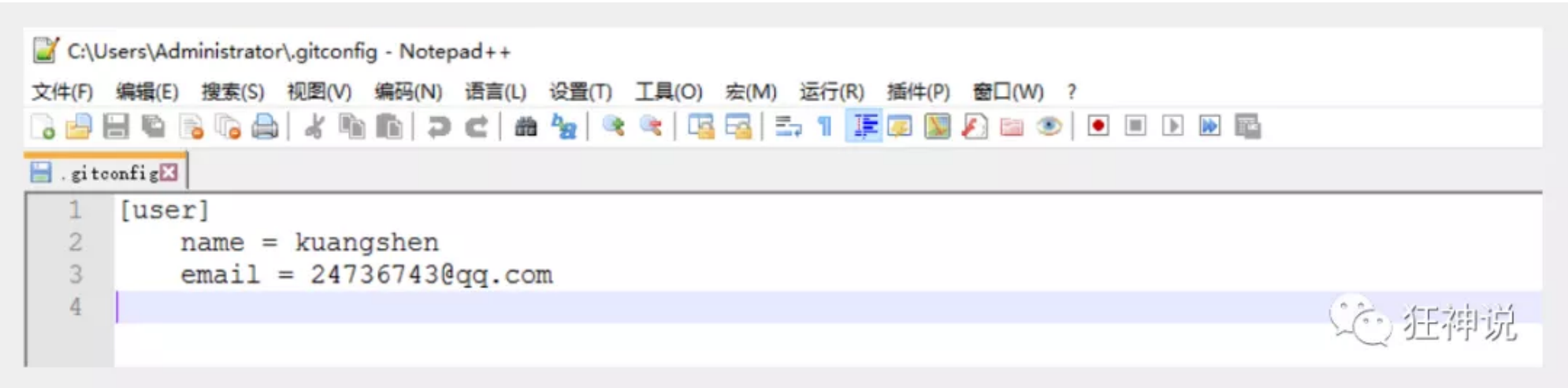
查看不同级别的配置文件：

```
#查看系统config
git config --system --list

#查看当前用户（global）配置
git config --global  --list
```

Git相关的配置文件：

1）、Git\etc\gitconfig ：Git 安装目录下的 `git config --system` 系统级

2）、C:\Users\Administrator\ .gitconfig 只适用于当前登录用户的配置 `--global` 全局



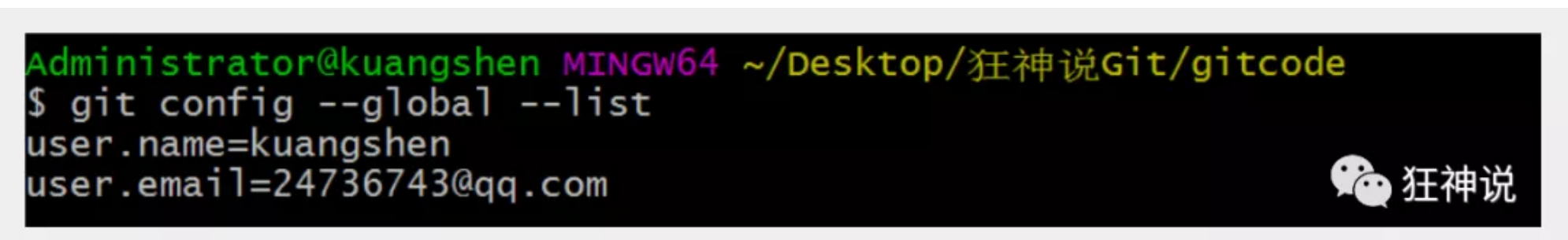
这里可以直接编辑配置文件，通过命令设置后会响应到这里。

设置用户名与邮箱（用户标识，必要）

当你安装Git后首先要做的事情是设置你的用户名称和e-mail地址。这是非常重要的，因为每次Git提交都会使用该信息。它被永远的嵌入到了你的提交中：

```
git config --global user.name "xxxx"  #名称
git config --global user.email xxxx@qq.com   #邮箱
```

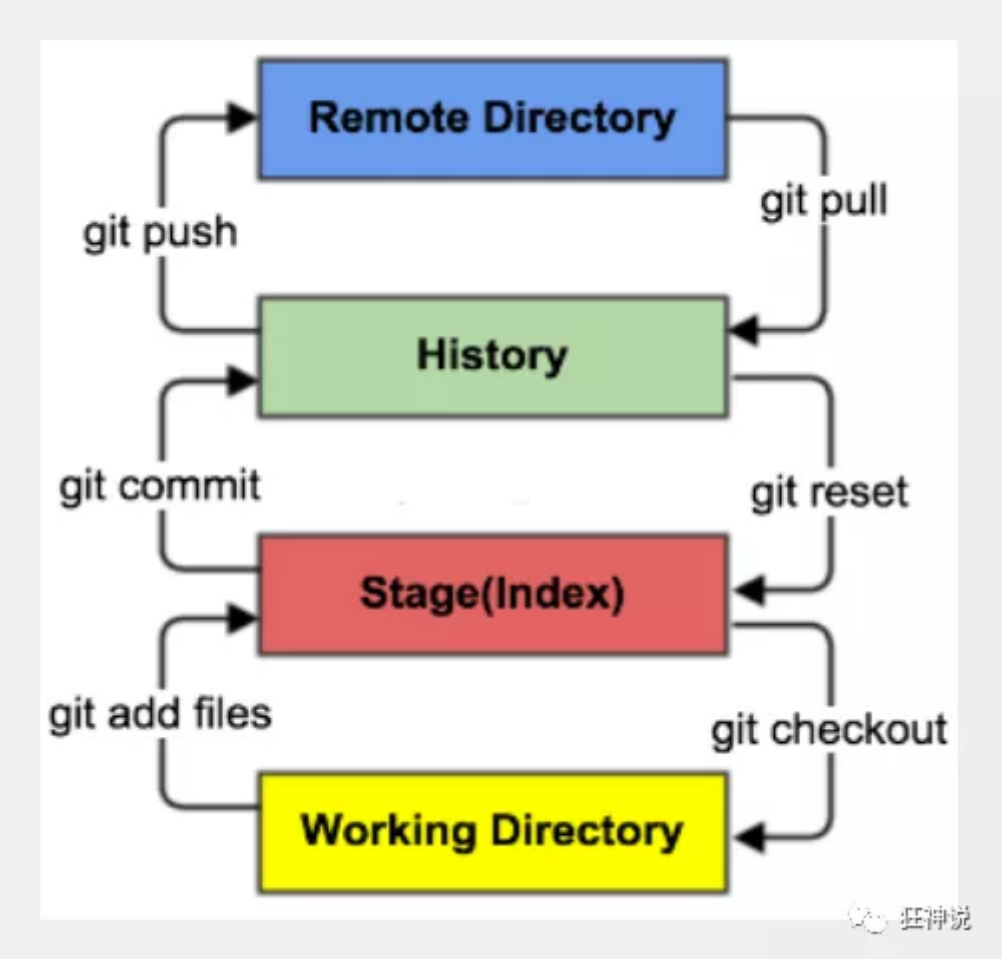
只需要做一次这个设置，如果你传递了--global 选项，因为Git将总是会使用该信息来处理你在系统中所做的一切操作。如果你希望在一个特定的项目中使用不同的名称或e-mail地址，你可以在该项目中运行该命令而不要--global选项。总之--global为全局配置，不加为某个项目的特定配置。



Git基本理论

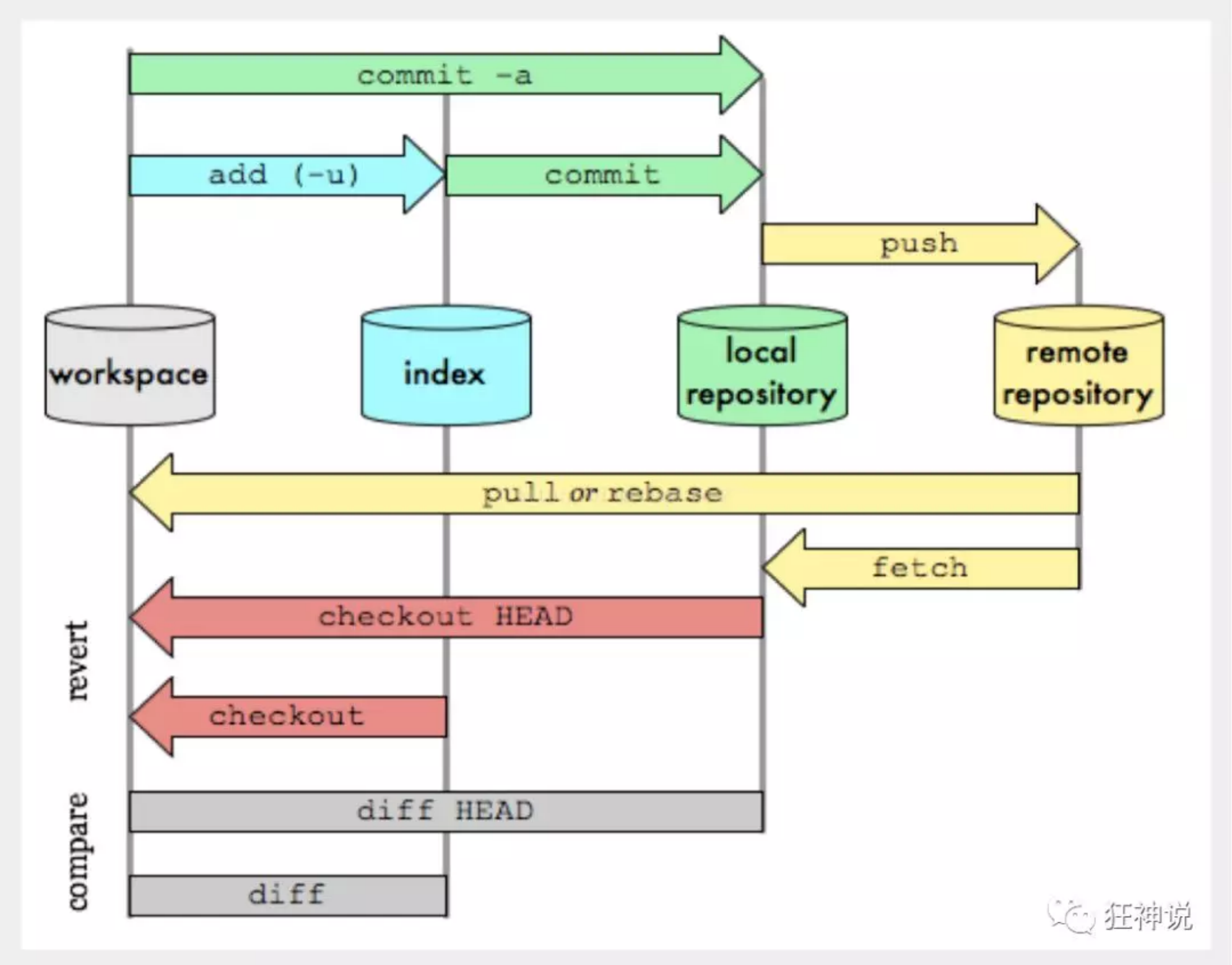
三个区域

Git本地有三个工作区域：工作目录（Working Directory）、暂存区(Stage/Index)、资源库(Repository或Git Directory)。如果在加上远程的git仓库(Remote Directory)就可以分为四个工作区域。文件在这四个区域之间的转换关系如下：



- Workspace：工作区，就是你平时存放项目代码的地方
- Index / Stage：暂存区，用于临时存放你的改动，事实上它只是一个文件，保存即将提交到文件列表信息
- Repository：仓库区（或本地仓库），就是安全存放数据的位置，这里面有你提交到所有版本的数据。其中HEAD指向最新放入仓库的版本
- Remote：远程仓库，托管代码的服务器，可以简单的认为是你项目组中的一台电脑用于远程数据交换

本地的三个区域确切的说应该是git仓库中HEAD指向的版本：



- Directory：使用Git管理的一个目录，也就是一个仓库，包含我们的工作空间和Git的管理空间。
- WorkSpace：需要通过Git进行版本控制的目录和文件，这些目录和文件组成了工作空间。
- .git：存放Git管理信息的目录，初始化仓库的时候自动创建。
- Index/Stage：暂存区，或者叫待提交更新区，在提交进入repo之前，我们可以把所有的更新放在暂存区。
- Local Repo：本地仓库，一个存放在本地的版本库；HEAD会只是当前的开发分支（branch）。
- Stash：隐藏，是一个工作状态保存栈，用于保存/恢复WorkSpace中的临时状态。

工作流程

git的工作流程一般是这样的：

1、在工作目录中添加、修改文件；

2、将需要进行版本管理的文件放入暂存区域；

3、将暂存区域的文件提交到git仓库。

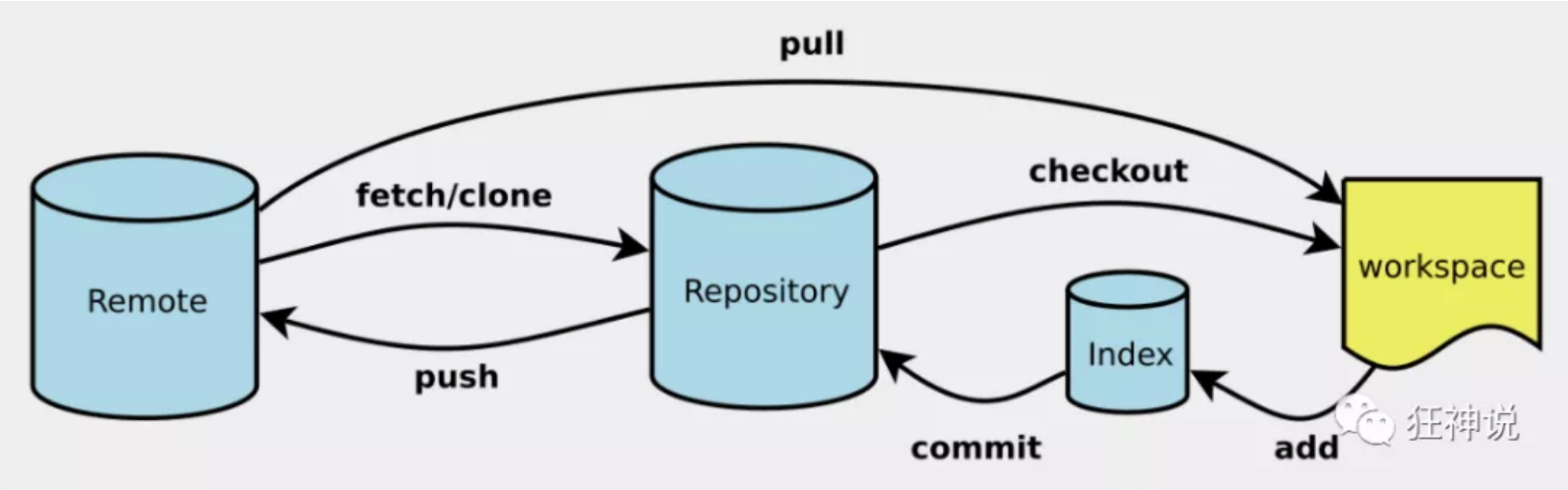
因此，git管理的文件有三种状态：已修改（modified）,已暂存（staged）,已提交(committed)

Git项目搭建

创建工作目录与常用指令

工作目录（WorkSpace)一般就是你希望Git帮助你管理的文件夹，可以是你项目的目录，也可以是一个空目录，建议不要有中文。

日常使用只要记住下图6个命令：



本地仓库搭建

创建本地仓库的方法有两种：一种是创建全新的仓库，另一种是克隆远程仓库。

- 创建全新的仓库，需要用GIT管理的项目的根目录执行：

```
# 在当前目录新建一个Git代码库
git init
```

- 执行后可以看到，仅仅在项目目录多出了一个.git目录，关于版本等的所有信息都在这个目录里面。

克隆远程仓库

- 另一种方式是克隆远程目录，由于是将远程服务器上的仓库完全镜像一份至本地！

```
# 克隆一个项目和它的整个代码历史(版本信息)
git clone [url]  # https://gitee.com/kuangstudy/openclass.git
```

- 去 gitee 或者 github 上克隆一个测试！

Git文件操作

文件的四种状态

版本控制就是对文件的版本控制，要对文件进行修改、提交等操作，首先要知道文件当前在什么状态，不然可能会提交了现在还不想提交的文件，或者要提交的文件没提交上。

- Untracked: 未跟踪, 此文件在文件夹中, 但并没有加入到git库, 不参与版本控制. 通过 `git add` 状态变为Staged.
- Unmodify: 文件已经入库, 未修改, 即版本库中的文件快照内容与文件夹中完全一致. 这种类型的文件有两种去处, 如果它被修改, 而变为Modified. 如果使用 `git rm` 移出版本库, 则成为Untracked文件
- Modified: 文件已修改, 仅仅是修改, 并没有进行其他的操作. 这个文件也有两个去处, 通过 `git add` 可进入暂存staged状态, 使用 `git checkout` 则丢弃修改过, 返回到unmodify状态, 这个 `git checkout` 即从库中取出文件, 覆盖当前修改！
- Staged: 暂存状态. 执行git commit则将修改同步到库中, 这时库中的文件和本地文件又变为一致, 文件为Unmodify状态. 执行git reset HEAD filename取消暂存, 文件状态为Modified

查看文件状态

上面说文件有4种状态，通过如下命令可以查看到文件的状态：

```
#查看指定文件状态
git status [filename]
#查看所有文件状态
git status


# git add .                    添加所有文件到暂存区
# git commit -m "消息内容"     提交暂存区中的内容到本地仓库 -m 提交信息
```

忽略文件

有些时候我们不想把某些文件纳入版本控制中，比如数据库文件，临时文件，设计文件等

在主目录下建立".gitignore"文件，此文件有如下规则：

- 忽略文件中的空行或以井号（#）开始的行将会被忽略。
- 可以使用Linux通配符。例如：星号（*）代表任意多个字符，问号（？）代表一个字符，方括号（[abc]）代表可选字符范围，大括号（{string1,string2,...}）代表可选的字符串等。
- 如果名称的最前面有一个感叹号（!），表示例外规则，将不被忽略。
- 如果名称的最前面是一个路径分隔符（/），表示要忽略的文件在此目录下，而子目录中的文件不忽略。

- 如果名称的最后面是一个路径分隔符（/），表示要忽略的是此目录下该名称的子目录，而非文件（默认文件或目录都忽略）。

```
#为注释
*.txt        #忽略所有 .txt结尾的文件,这样的话上传就不会被选中!
!lib.txt     #但lib.txt除外
/temp        #仅忽略项目根目录下的TODO文件,不包括其它目录temp
build/       #忽略build/目录下的所有文件
doc/*.txt    #会忽略 doc/notes.txt 但不包括 doc/server/arch.txt
```

使用Gitee

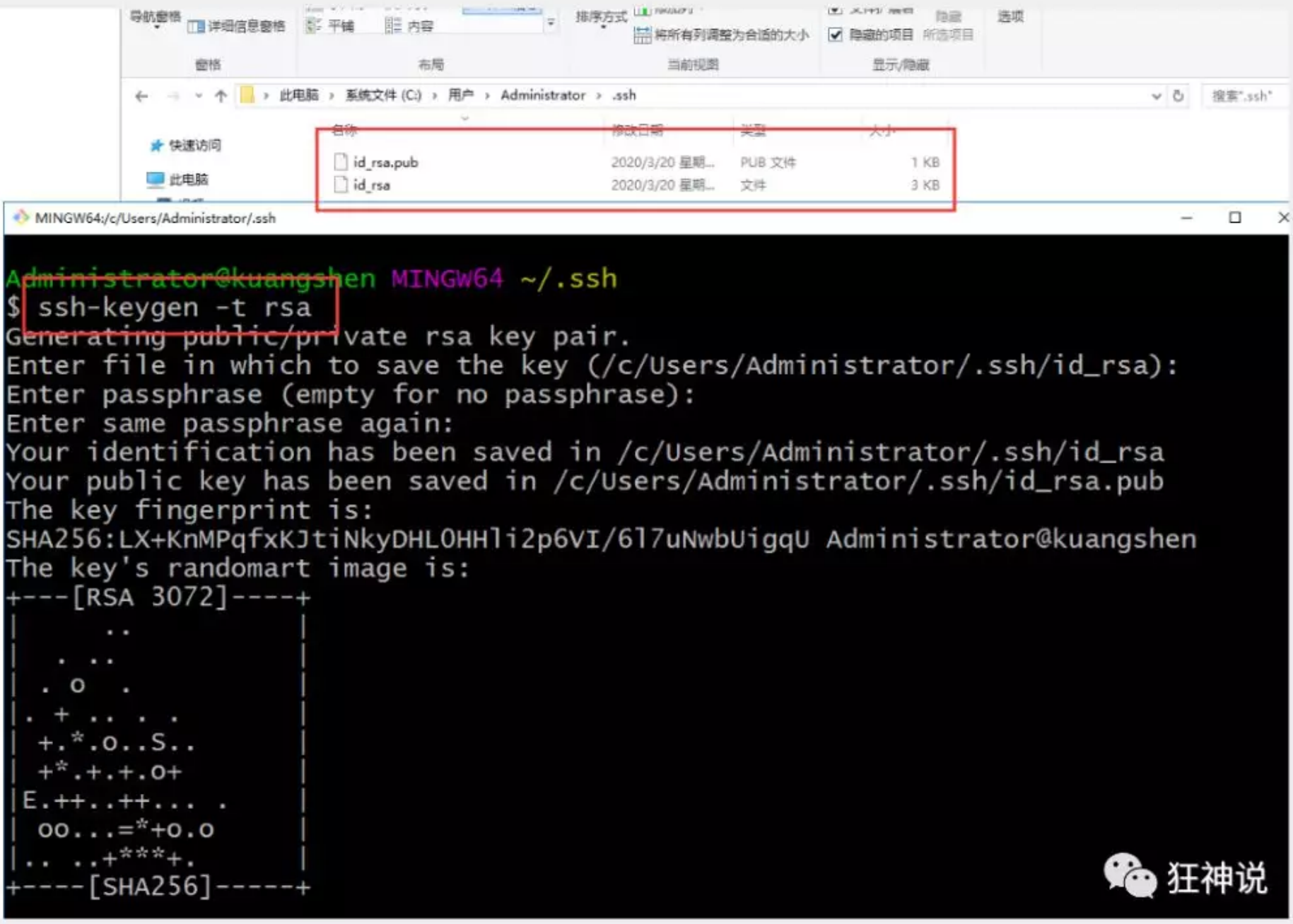
github 是有墙的，比较慢，在国内的话，我们一般使用 gitee ，公司中有时候会搭建自己的gitlab服务器

这个其实可以作为大家未来找工作的一个重要信息！

- 注册登录码云，完善个人信息

- 设置本机绑定SSH公钥，实现免密码登录！（免密码登录，这一步挺重要的，码云是远程仓库，我们是平时工作在本地仓库！）

```
# 进入 C:\Users\Administrator\.ssh 目录
# 生成公钥
ssh-keygen
```



- 将公钥信息public key 添加到码云账户中即可！

- 使用码云创建一个自己的仓库！

仓库名称 ✔

gitstudy

归属 路径

狂神说 / gitstudy

仓库地址：https://gitee.com/kuangstudy/gitstudy

仓库介绍 非必填

狂神聊Git，一定记得三连投币，然后关注公众号：狂神说

是否开源

私有 公开

任何人都可以访问该仓库的代码和其他任何形式的资源

选择语言 添加 .gitignore 添加开源许可证

Java Java GPL-3.0

使用Readme文件初始化这个仓库

使用Issue模板文件初始化这个仓库

使用Pull Request模板文件初始化这个仓库

选择分支模型 (仓库初始化后将根据所选分支模型创建分支)

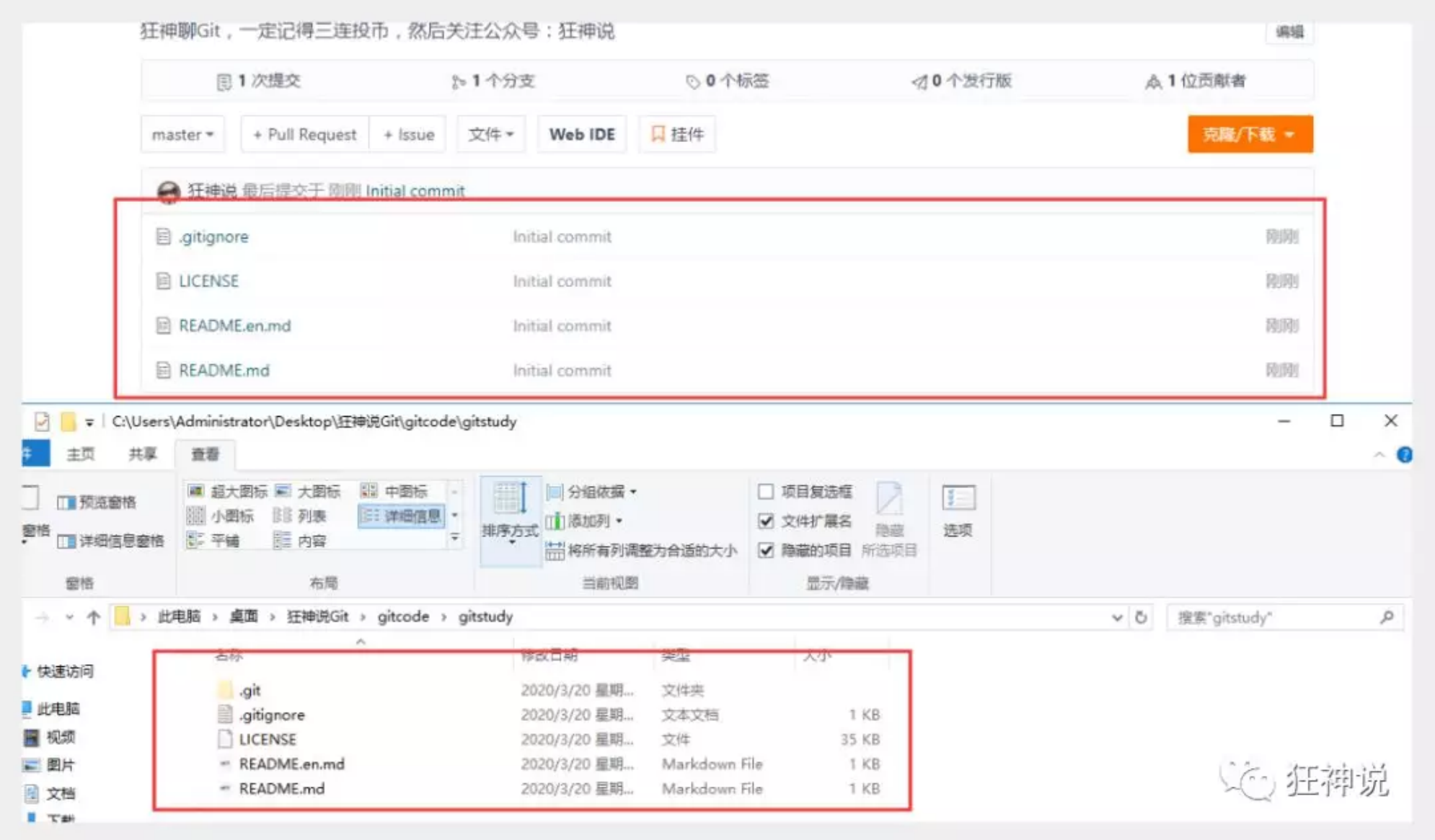
单分支模型 (只创建 master 分支)

导入已有仓库

创建

狂神说

- 克隆到本地！



使用Github

基本操作和gitee差不多

- 删除已经存在的.ssh 目录:

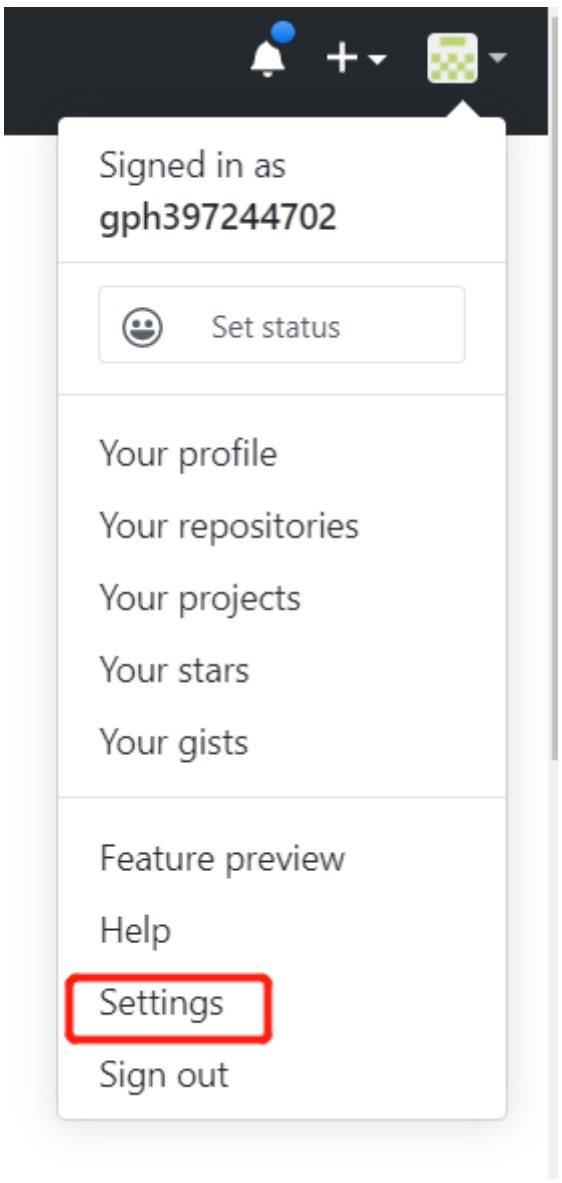
```
rm -r .ssh
```

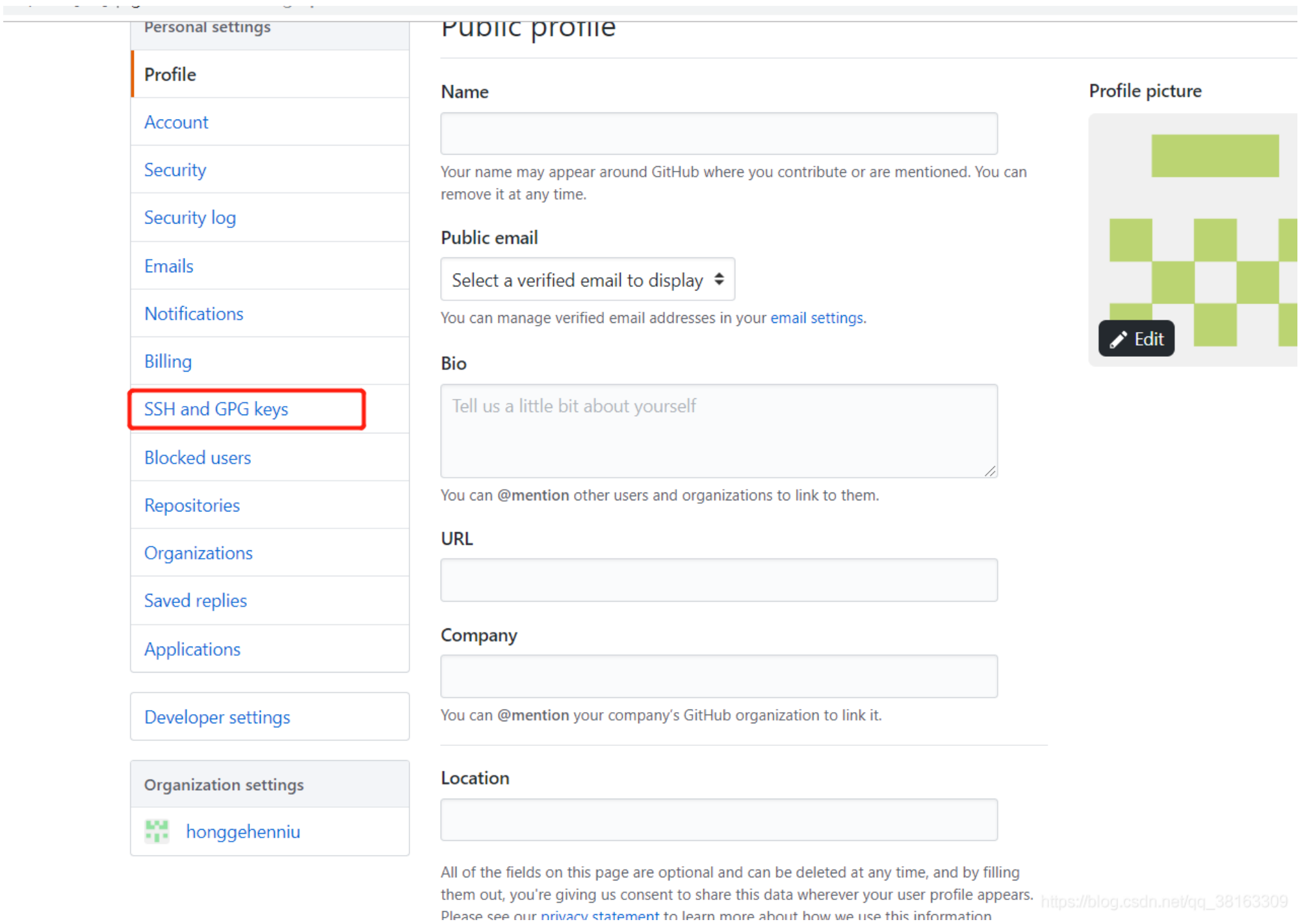
- 生成密钥对

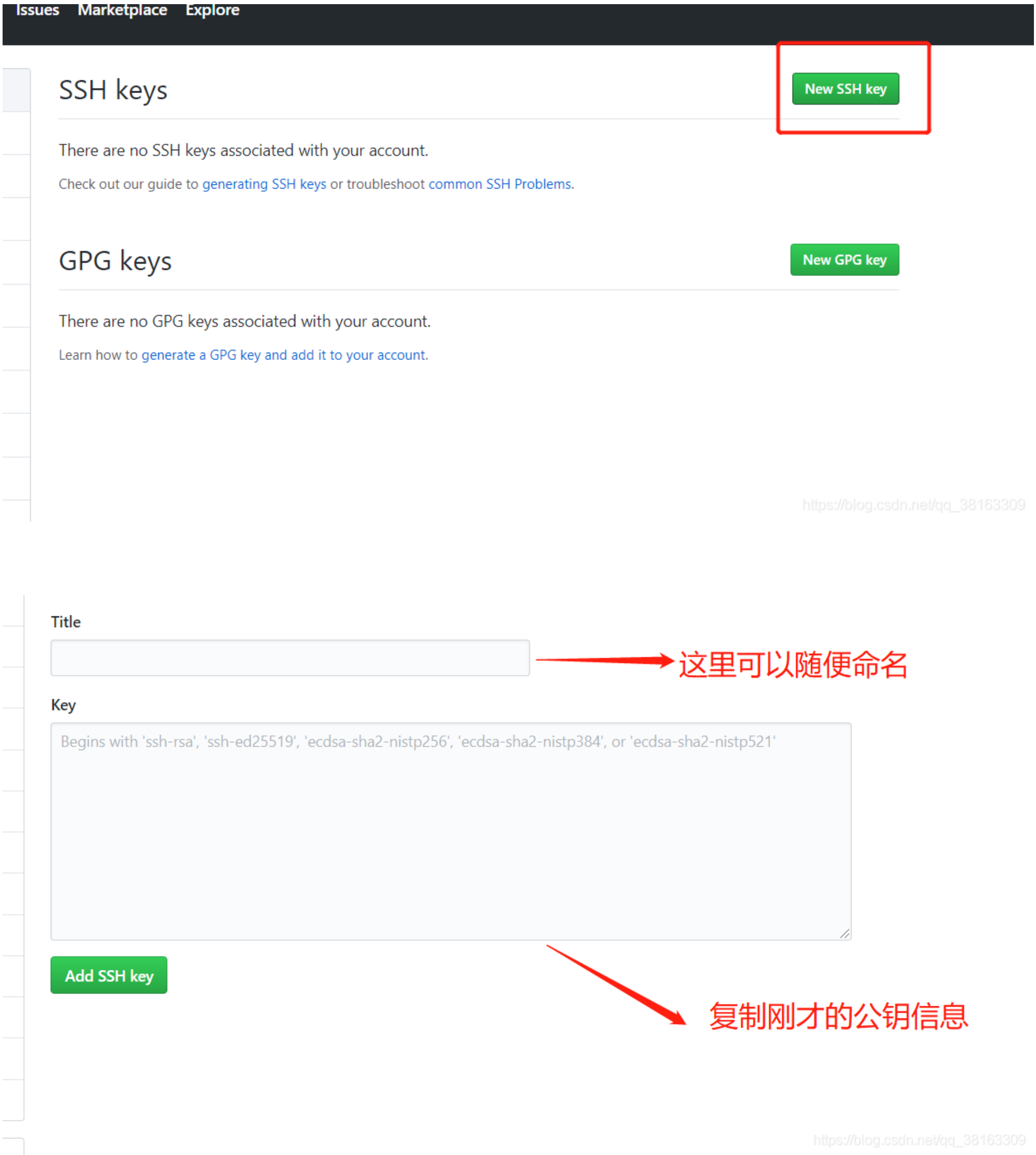
```
# 进入 C:\Users\Administrator\.ssh 目录
# 生成公钥
ssh-keygen -t rsa -C "xxxx@youremail.com"

# 然后一直按回车键就行了
```

- 查看.ssh文件目录

- 复制 id_rsa.pub 文件内容，登录 GitHub，点击用户头像→Settings→SSHandGPG keys


Signed in as
gph397244702
Set status
Your profile
Your repositories
Your projects
Your stars
Your gists
Feature preview
Help
Settings
Sign out


Personal settings
Profile
Account
Security
Security log
Emails
Notifications
Billing
SSH and GPG keys
Blocked users
Repositories
Organizations
Saved replies
Applications
Developer settings
Organization settings
honggehenniu
Public profile
Name
Your name may appear around GitHub where you contribute or are mentioned. You can remove it at any time.
Public email
Select a verified email to display
You can manage verified email addresses in your email settings.
Bio
Tell us a little bit about yourself
You can @mention other users and organizations to link to them.
URL
Company
You can @mention your company's GitHub organization to link it.
Location
All of the fields on this page are optional and can be deleted at any time, and by filling them out, you're giving us consent to share this data wherever your user profile appears. Please see our privacy statement to learn more about how we use this information.
Profile picture
Edit
https://blog.csdn.net/qq_38163309



Git基本操作

常用命令

命令	功能	格式	参数说明	使用实例
ssh –keygen –t rsa	生成密钥	ssh –keygen –t rsa –C [email]	email：邮箱地址	在C盘.ssh文件夹下获取密钥文件“id_rsa.pub”ssh –keygen –t rsa –C "devcloud_key01@huawei.com"
git branch	新建分支。	git branch [new branchname]	new branchname：新的分支名	新建分支：git branch newbranch
git branch –D	删除分支	git branch –D [new branchname]	new branchname：新的分支名	删除本地分支：git branch –D newbranch删除服务器仓库分支git branch –rd origin/newbranch同步远端已删除的分支git remote prune origin
git add	添加文件到暂存区。	git add [filename]	filename：文件名	添加一个文件到暂缓区：git add filename添加所有修改的和新增的文件到暂缓区：git add .
git rm	删除本地目录或文件。	git rm [filename]	filename：文件名或目录名	删除文件：git rm filename
git clone	克隆远程仓库。	git clone [VersionAddress]	VersionAddress：版本库的网址。	克隆**jQuery**的版本库：git clone https://github.com/jquery/jquery.git该命令会在本地主机生成一个目录，与远程主机的版本库同名。
git pull	把远程仓库的分支pull到本地，再与本地的指定分支合并。	git pull [RemoteHostname] [RemoteBranchname]: [LocalBranchname]	-	取回**“origin”**主机的**“next”**分支，与本地的**“master”**分支合并：git pull origin next:master
git diff	文件、分支、目录或版本的比较。	git diff	-	当前与**“master”**分支的比较：git diff master
git commit	文件提交。	git commit	-	添加提交信息：git commit –m "commit message"
git push	推送文件到远程仓库。	git push [RemoteHostname] [LocalBranchname] [RemoteBranchname]	-	如果省略远程分支名，则表示将本地分支推送与之存在“追踪关系”的远程分支（通常两者同名），如果该远程分支不存在，则会被新建：git push origin master上面命令表示，将本地的master分支推送到origin主机的master分支。如果后者不存在，则会被新建。
git merge	合并分支。	git merge [branch]	branch：分支名	假设当前分支为“develop”，将master主分支之后的最新提交merge到当前的develop分支上：git merge master
git checkout	切换分支。	git checkout [branchname]	branchname：分支名	切换到master分支：git checkout master
git log	列出日志信息。	git log	-	列出所有的log：git log –-all
git status	查看状态输出。	git status	-	git status
git grep	查找字符串。	git grep	-	查找是否有**“hello”**字符串：git grep "hello"
git show	显示内容或修改的内容。	git show	-	git show v1显示**“tag v1”**的修改内容git show HEAD显示当前版本的修改文件git show HEAD^显示前一版本所有的修改文件git show HEAD~4显示前4版本的修改文件
git stash	暂存区。	git stash	-	git stash list列出暂存区的文件git stash pop取出最新的一笔，并移除git stash apply取出但不移除git stash clear清除暂存区
git ls-files	查看文件。	git ls-files	-	git ls-files –d查看已经删除的文件git ls-files –d |xargs git checkout将已删除的文件还原
git remote	操作远程。	git remote	-	git push origin master:newbranch增加远程仓库的分支git remote add newbranch增加远程仓库的分支git remote show列出现在远程有多少版本库git remote rm newbranch删除远程仓库的新分支git remote update更新远程所有版本的分支

创建仓库命令

下表列出了 git 创建仓库的命令：

命令	说明
`git init`	初始化仓库
`git clone`	拷贝一份远程仓库，也就是下载一个项目。

提交与修改

Git 的工作就是创建和保存你的项目的快照及与之后的快照进行对比。

下表列出了有关创建与提交你的项目的快照的命令：

命令	说明
`git add`	添加文件到仓库
`git status`	查看仓库当前的状态，显示有变更的文件。
`git diff`	比较文件的不同，即暂存区和工作区的差异。
`git commit`	提交暂存区到本地仓库。
`git reset`	回退版本。
`git rm`	删除工作区文件。
`git mv`	移动或重命名工作区文件。

提交日志

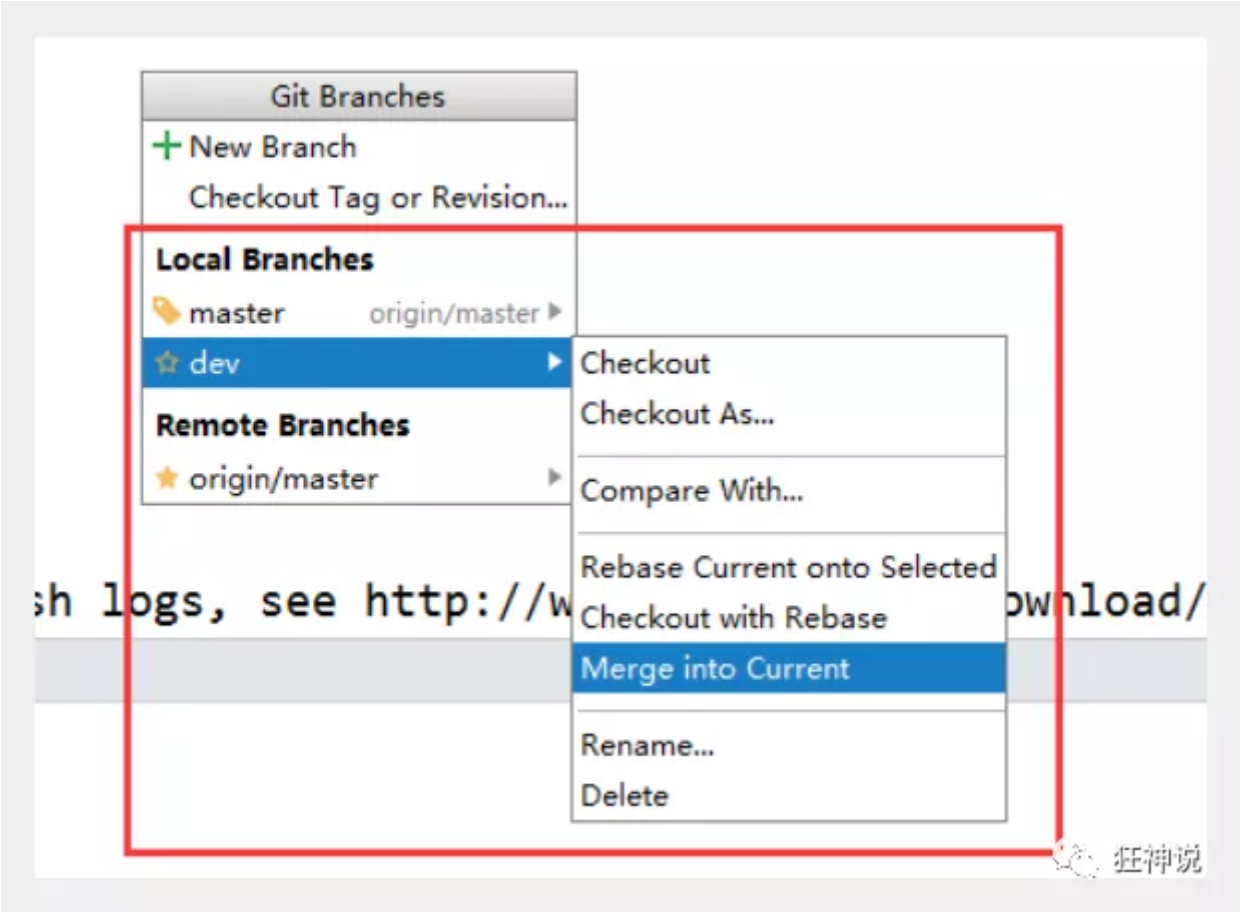
命令	说明
`git log`	查看历史提交记录
`git blame <file>`	以列表形式查看指定文件的历史修改记录

远程操作

命令	说明
`git remote`	远程仓库操作
`git fetch`	从远程获取代码库
`git pull`	下载远程代码并合并
`git push`	上传远程代码并合并

说明：Git分支

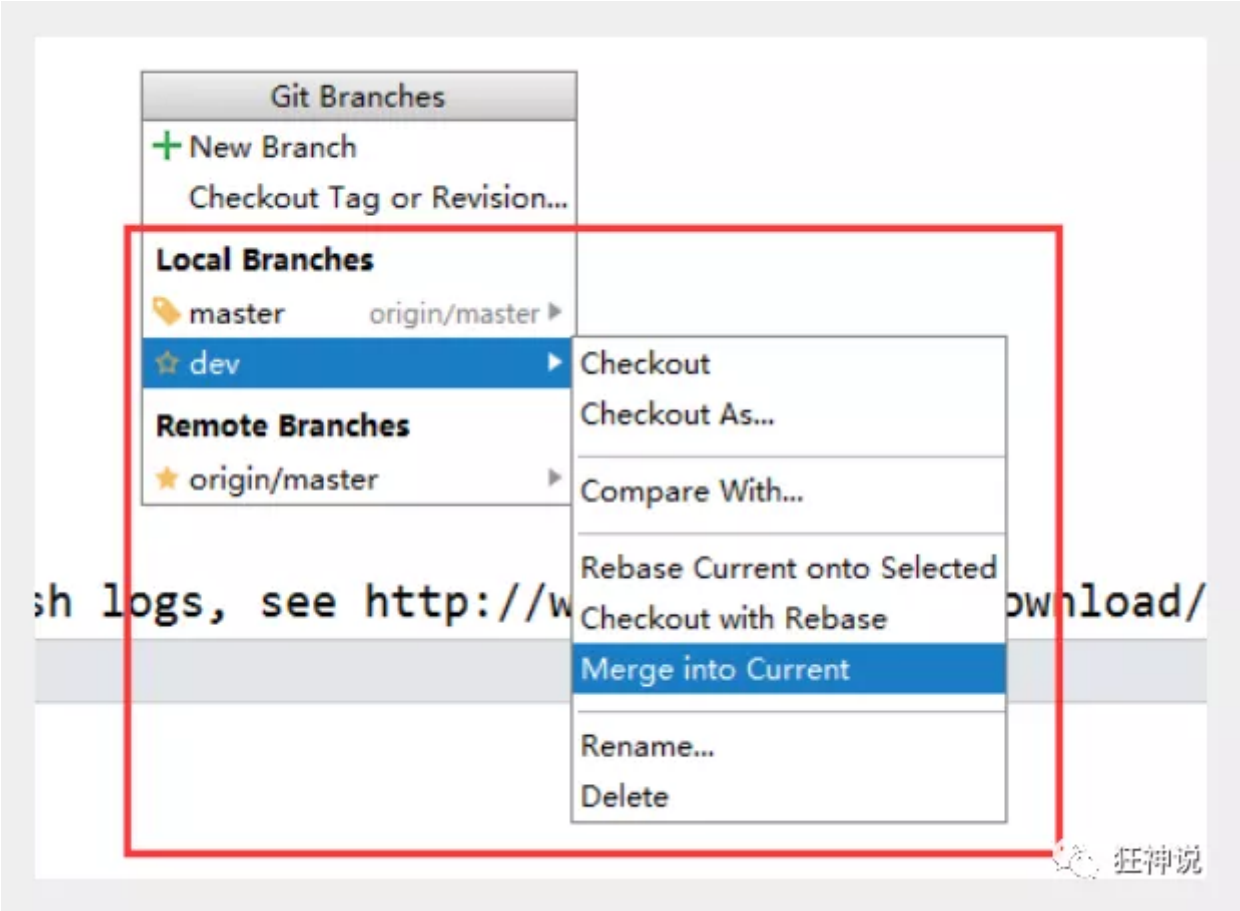
分支在GIT中相对较难，分支就是科幻电影里面的平行宇宙，如果两个平行宇宙互不干扰，那对现在的你也没啥影响。不过，在某个时间点，两个平行宇宙合并了，我们就需要处理一些问题了！



git分支中常用指令：

```
# 列出所有本地分支
git branch

# 列出所有远程分支
git branch -r

# 新建一个分支，但依然停留在当前分支
git branch [branch-name]

# 新建一个分支，并切换到该分支
git checkout -b [branch]

# 合并指定分支到当前分支
$ git merge [branch]

# 删除分支
$ git branch -d [branch-name]

# 删除远程分支
$ git push origin --delete [branch-name]
$ git branch -dr [remote/branch]
```

IDEA中集成Git



如果同一个文件在合并分支时都被修改了则会引起冲突：解决的办法是我们可以修改冲突文件后重新提交！选择要保留他的代码还是你的代码！

master主分支应该非常稳定，用来发布新版本，一般情况下不允许在上面工作，工作一般情况下在新建的dev分支上工作，工作完后，比如上要发布，或者说dev分支代码稳定后可以合并到主分支master上来。